RO · Modelado · base ...

Hernandez Crisostomo Emilio Corrección de examen Parcial mPOO

Pregunta 0①

Contesta con tus palabras. Puedes apoyarte en un ejemplo. ¿Qué hace new al crear un objeto? Explica qué recibe la variable a la que asignas la expresión

R= Al utilizar new, se crea la instancia del objeto y se ejecuta el constructor para inicializarlo, mientras que la variable asignada recibe una referencia directo a ese objeto recien creado.

Pregunta 0.2

¿Que significa void en la cabecera de un método? Explica si ese metodo puede modificar un atributo.

R= El void en la cabecera significa que el método no devuelve ningún valor con return. Sí puede modificar un atributo, ya que el metodo actúa directamente sobre el estado del objeto aunque no entregue un resultado de entorno.

Pregunta 0.3

¿Para qué sirve this? Explica cómo distingue un atributo de un Parametro que tiene el mismo nombre

R= El this sirve como una referencia al objeto actual, permitiendo acceder a sus elementos dentro del mismo valor u objeto. (Por ejemplo con this.Papas). Distingue un atributo de un Parametro con el mismo nombre porque en una asignación como this.x = x, lo que está a la izquierda con this es el atributo del objeto y a la derecha es el parámetro recibido.

Pregunta 0.4

Java siempre pasa por valor. ¿Qué recibe un método cuando le pasas un Int y cuando le pasas un arreglo.

R= Cuando le pasas un Int, el método recibe una copia del numero primitivo de modo que modificar ese elemento dentro del método no afecta a la variable original. Cuando pasa un arreglo, el método recibe una copia de la referencia a donde va ese arreglo en memoria. Aunque reasignar el Parametro por completo no cambia la variable de quien llama.

Pregunta 0.5

¿En qué se distinguen JDK y JRE?

Explica cual necesitas en el Programa y por que?

El JRE es el entorno de ejecución que incluye la JVM y sus bibliotecas, mientras que el JDK es el kit de desarrollo que añade las herramientas necesarias para compilar. como el comando javac. Necesitamos el JDK para compilar un Programa por que contiene herramientas indispensables para traducir el código fuente.

R1 ¿Qué imprime?

R= escribe la salida exacta, una línea por renglón

```
Integer base = 5;
Integer copia = base;
Int [] aumentos = {1,2,3};
  for (Int aumento: aumentos) {
      base = base + aumento;
      System.out.Println (base /2);
}
String etiqueta = "D" + copia +1;
System.out.Println(etiqueta);
System.out.Println((double) (base/2));
```

R=	aumento	base	copia
Inicio	—	5	5
vuelta 1	1	6	5
2	2	8	5
3	3	11	5
etiqueta	—	11	5
Final	—	11	5

R= Se imprime 3, 4, 5, D51 Porque en el ciclo for la división se realiza entre enteros (Int/Int), por lo que en la ultima vuelta 11/2 da como resultado 5, y al aplicarle el cast a (double) se convierte en 5.0. Además, la variable etiqueta concatena el texto con las variables obteniendo D51.

R2 a) ¿Compila, truena o funciona? R= no compila

```
IF (true) { Int voltaje = 12; }
System.out.Println (voltaje);
```

El Fragmento no compila Porque la variable voltaje se declara dentro del bloque del IF ({Int voltaje = 12;}), Por lo que tiene un ámbito local y no existe fuera de él. Aunque la condición sea siempre true, las llaves delimitan el alcance de la variable haciendo que la impresión falle al intentar acceder a una variable que no existe.

R2 b) ¿compila, truena o funciona?

```
StringBuilder ruta = new StringBuider ("Q");
ruta.append (2+3);
System.out.Println(ruta);
```

R= Funciona

R= El Fragmento del código funciona e imprime Q5, esto ocurre Por que se inicializa un objeto StringBuilder con la cadena "Q", y Posteriormente la instrucción ruta.append (2+3) evalua Primero la Suma aritmetica 2+3 = 5. dicho resultado Se concatena.

R2 c) ¿compila, truena o funciona? R= Compila Pero truena

```
Int [] canales = null;
System.out.Println (canales.length);
```

R= El fragmento Si compila Pero truena en ejecución al llegar a la línea System.out.Println(canales.length) la regla de Java que lo Provoca es que la variable canales tiene un valor NULL y no apunta a ningún arreglo en memoria. Al intentar consultar la Propiedad de una referencia Nula y lanza el error

R3. Modelado: base, reto y justificación

P= Cada Termostato guarda etiqueta fija y temperatura. El rango común del edificio es de 16 a 28 grados. Constructor y ajustar(int temp) reciben valores válidos; ajustar asigna. Añade una consulta de temperatura base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final ¿Cómo declaras los dos extremos del rango para que sean comunes y no cambien? Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

R=

```
class Termostato {
    final String etiqueta;
    int temperatura;
    static final int minimo = 16;
    static final int maximo = 28;

  Termostato(String etiqueta, int
  temperatura) {
        this.etiqueta = etiqueta;
        this.temperatura = temperatura;
        }
    void ajustar(int temperatura){
        this.temperatura = temperatura;
        }
    int consultar(){
        return this.temperatura;
        }
}
```

Justificación

1. Atributo etiqueta: Tipo string de instancia
2. Atributo temperatura: Tipo int. de instancia, mutable, ya que el termostato permite ajustar y cambiar su valor.
3. Constantes del rango max min
4. Constructor: recibe los parametros necesarios para inicializar su atributos
5. Metodos (ajustar y consultar)

R4 Código con repetición

Devuelve el promedio entero de los valores pares de Pulso usando for-each

static int PromedioPares(int[] pulsos)

{3,8,2,5} → 5 · {1,3,5} → 0 {-4,3,6} →

entrada		filtra los pares
{3,8,2,5}	5	8 y 2 8+2=10/2=5
{1,3,5}	0	=0 Sin pares
{-4,3,6}	7	-4,6 -4+6=2/2=7

R= static int PromedioPares(int[] Pulsos)

```
{
  int Suma=0;
  int Contador=0;
  for(int Pulso: Pulsos){
    IF(Pulso % 2==0){
      Suma += Pulso;
      Contador ++;
    }
  }
  IF(Contador == 0){
    return 0;
  }
  return Suma/Contador;
}
```

R5. Código con Selección

Static String estadoTermico (Int temperatura)

<0 >120 Invalida

0-29 Fria

30-59 templada

60-89 Caliente

90-120 muy caliente

Temp	Iteracion
-5	Invalida
0-29	Fria
45	Templada
75	Caliente
100	Muy caliente

R=

```
Static String estadoTermico (Int tempera
    IF (temperatura <0 || temperatura >
        return "Invalida";
    }else IF (temp <=29 ){
        return "Fria";
    }else IF (temp <=59 ){
        return " templada";
    }else IF (temp <=89) {
        return "Caliente";
    }else {
        return "muy caliente"
    }
}
```